EX 1 : THREE-LAYER NEURAL NETWORK FROM SCRATCH

AIM:

To Design and implement a three-layer neural network from scratch using Python. Train the network using the backpropagation algorithm with appropriate activation and loss functions. Apply the model to recognize handwritten digits using the MNIST dataset.

OBJECTIVES:

1. Understand the structure and working of a basic feedforward neural network.
2. Implement forward propagation and backpropagation from scratch.
3. Use the sigmoid activation function and cross-entropy loss.
4. Train the model on the MNIST dataset and evaluate the loss over epochs.
5. Visualize training performance and perform predictions on sample images.

ALGORITHM:

Step 1: Import Necessary Libraries

Import NumPy for mathematical operations, Matplotlib for visualization, TensorFlow for loading the MNIST dataset, and OneHotEncoder for encoding labels.

Step 2: Load and Preprocess the Dataset

Load the MNIST handwritten digit dataset, normalize pixel values to the range 0–1, flatten each image into a 784-element vector, and one-hot encode the digit labels.

Step 3: Initialize the Network

Create a three-layer neural network consisting of an input layer with 784 neurons, a hidden layer, and an output layer with 10 neurons. Initialize weights and biases with small random values.

Step 4: Define Activation and Loss Functions

Use the sigmoid activation function in the hidden and output layers. Define categorical cross-entropy loss to measure the difference between predicted outputs and actual labels.

Step 5: Perform Forward Propagation

Pass the input images through the network, calculate the weighted sums, apply the activation functions, and obtain the predicted digit probabilities.

Step 6: Perform Backpropagation

Calculate the output error, compute gradients of weights and biases using the chain rule, and propagate the errors backward through the network.

Step 7: Update Weights and

Update the weights and biases using gradient descent and a suitable learning rate to

minimize the loss.

Step 8: Train the Model

Repeat forward propagation, loss calculation, backpropagation, and parameter updates for multiple epochs using the training dataset.

Step 9: Visualize Training Loss

Plot the training loss against the number of epochs using Matplotlib to observe the learning performance.

Step 10: Predict and Evaluate

Use the trained network to predict handwritten digits from sample test images, compare predictions with actual labels, and evaluate the model's accuracy.

SOURCE CODE:

```
# Step 1: Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from PIL import Image

# Step 2: Load the MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Step 3: Preprocess the Dataset
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

x_train = x_train.reshape(-1, 28 * 28)
x_test = x_test.reshape(-1, 28 * 28)

# Step 4: Build the Neural Network
model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

# Step 5: Compile the Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Step 6: Train the Model
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)
```

```
# Step 7: Evaluate the Model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print("Test Loss:", test_loss)
print(f"Test Accuracy: {test_acc:.4f}")

# Step 8: Plot Training Loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()

# Step 9: Predict Sample Test Images
predictions = model.predict(x_test[:5], verbose=0)

for i in range(5):
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(
        f"Predicted: {np.argmax(predictions[i])}, "
        f"Actual: {y_test[i]}"
    )
    plt.axis('off')
    plt.show()

# Step 10: Predict Your Own Handwritten Image
# Upload/save your image as user_handwritten_image.png
try:
    user_image = Image.open(
        'user_handwritten_image.png'
    ).convert('L')

    user_image = user_image.resize((28, 28))

    image_array = np.array(user_image).astype('float32') / 255.0
    image_array = image_array.reshape(1, 784)

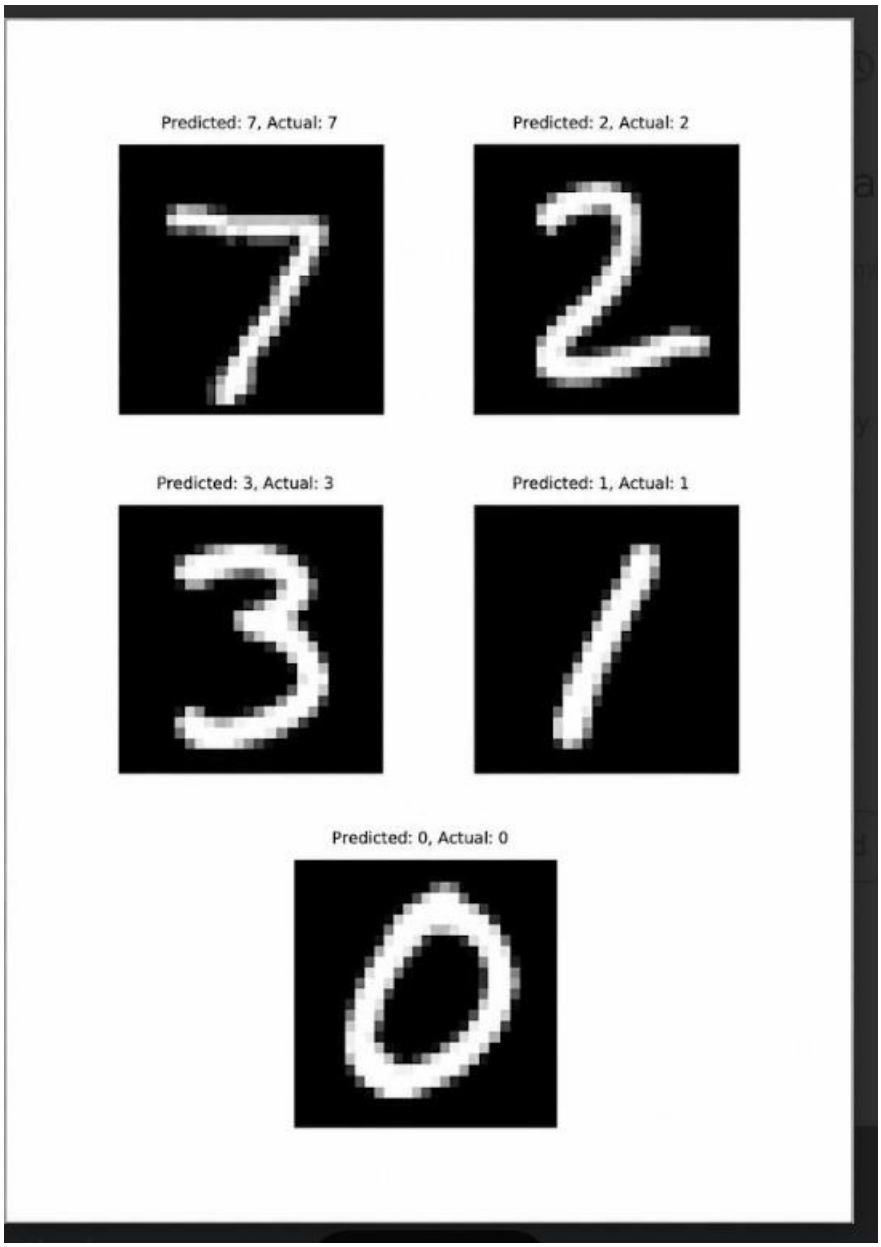
    prediction = model.predict(image_array, verbose=0)
    predicted_digit = np.argmax(prediction[0])

    plt.imshow(user_image, cmap='gray')
    plt.title(f'Predicted Digit: {predicted_digit}')
    plt.axis('off')
    plt.show()

    print("Predicted Digit:", predicted_digit)

except FileNotFoundError:
    print("Custom image not found. Add the image to the working folder.")
```

OUTPUT:



RESULT:

This experiment successfully demonstrates the manual implementation of a basic three-layer neural network. Through backpropagation and weight updates using gradient descent, the model learns to classify handwritten digits. It emphasizes key deep learning concepts such as feedforward computation, loss minimization, and numerical stability in gradient calculations